

Claude

Friday, October 9, 2026 · CLAUDE

https://github.com/SETAPESU26/03_fishing this is my repo for this project i gotta work on

Let me look at the repo and its README first.

Got the README. Let me pull the code down and see what's there.

Cloned it and read through everything. It's small (~230 lines), so this is doable in the 60 min.

Where each task lands:

- **Task 1 (bug):** `catch.py` only compares `hook.y` to `fish.y`, so x never matters. Both objects already have `get_rect()`, so what could you use instead of a distance check?
- **Task 2 (fish types):** `Fish` already takes speed, size, color and points. The three fish in `game_engine.py` are just hardcoded identical ones.
- **Task 3 (casting):** `update()` in `game_engine.py` calls `start_cast()` whenever the hook is IDLE. That's the auto-loop. `main.py` also never handles key events.
- **Task 4 (timer):** nothing exists yet. The engine needs a round state, and `renderer.draw_banner` is already there for the end screen.

Two things I can't do for you: record the before/after videos, and push to your GitHub account. Both are on your end, and the repo is public so you can fork it and clone.

How do you want to run this? I can write the code task by task as a patch you apply and commit one at a time, or you can try each task first and I review. Either way this chat becomes your exported history.

📄 🔄 👍 🗑️ ↺ ...

so how i start tell me steps one by one

Start with setup. Do these in order:

1. **Fork the repo.** On GitHub open `SETAPESU26/03_fishing` and click **Fork** to your personal account. Don't open a PR back to the main repo.
2. **Clone your fork** (use your username):

```
git clone https://github.com/<your-username>/03_fishing.git
cd 03_fishing/fishing
```

1. **Install and run:**

```
pip install -r requirements.txt
python main.py
```

1. **Record the "before" video.** Capture 10 seconds of the game as it is now. Watch for a fish getting caught while it's nowhere near the hook. That's the bug.

Then make a `Lab-4` folder in the repo root for the videos, code and chat export later.

Tell me when it runs and what you saw in the video. Then we'll start Task 1, and I'll ask you what the bug actually is first.



ok recording done what next

Task 1 now. Quick question first: what did you see in the video? In your own words, why did a fish get caught when it wasn't touching the hook?

Look at `check_catch` in `game/catch.py` and ask yourself two things:

- Which coordinate is it comparing, and which one is missing?
- Both `hook` and `fish` have a `get_rect()`. What does pygame offer for checking whether two rects overlap?

Answer those two and I'll write the fix. Then you test it and commit it as its own commit.



Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the catch-detection bug

A fish is supposed to be caught only when the hook actually overlaps it. In the current build, `check_catch` (in `game/catch.py`) only compares the hook's depth to the fish's depth, within a fixed 10-pixel tolerance - it never compares horizontal position at all. That means a fish can register as caught while it's still far off to one side of the screen, just because the hook happens to be passing through its depth. Fix the check so it's based on the hook and fish actually overlapping, not just being at a similar depth.

Task 2: Implement multiple fish types

Introduce at least two fish types that differ in movement speed and point value (for example, a slow low-value fish and a fast high-value one).

Give each type its own look (color and/or size) so they're visually distinguishable, and make sure the correct point value is awarded when each type is caught.

Task 3: Implement player-controlled casting

Change the hook so the player decides when it casts, instead of it looping automatically. Pressing a key should start a cast if the hook is currently idle; it should then travel down and return on its own (already handled) once it reaches maximum depth or catches a fish. A new cast should not be able to interrupt one that's already in progress.

Task 4: Implement a 30-second round timer

Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero, no further catches should be possible, the round should end, and the final score should be shown clearly.

Provide a way to start a new round with the score and timer both reset.

you only tell me the fix

Replace the body of `check_catch` in `game/catch.py`:

python

```
def check_catch(hook, fish_list):
    """
    Returns the fish the hook has caught, or None.
    """
    hook_rect = hook.get_rect()
    for fish in fish_list:
        if hook_rect.colliderect(fish.get_rect()):
            return fish
    return None
```

It now checks that the two rectangles actually overlap, so x and y both count. The old 10px depth check is gone.

Test: run `python main.py`. A fish should only get hooked when it passes through the hook's column, and fish far off to the side should be ignored.

Commit:

```
git add fishing/game/catch.py
git commit -m "Task 1: fix catch detection to use rect overlap"
```

Tell me when it's committed, and we'll do Task 2.

Two edits.

1. `game/fish.py` — add this at the bottom:

python

```
FISH_TYPES = {
    "slow": {"speed": 2, "point_value": 10, "width": 40, "height": 20,
"color": (80, 180, 220)},
    "fast": {"speed": 5, "point_value": 30, "width": 24, "height": 12,
"color": (240, 200, 40)},
}

def make_fish(kind, x, y, direction=1):
    cfg = FISH_TYPES[kind]
    return Fish(
        x=x, y=y,
        speed=cfg["speed"] * direction,
        width=cfg["width"], height=cfg["height"],
        point_value=cfg["point_value"], color=cfg["color"],
    )
```

2. `game/game_engine.py`

Change the import:

python

```
from game.fish import make_fish
```

Replace the `self.fish_list` in `__init__`:

python

```
self.fish_list = [
    make_fish("slow", x=100, y=180),
    make_fish("fast", x=550, y=230, direction=-1),
    make_fish("slow", x=400, y=280, direction=-1),
    make_fish("fast", x=250, y=340),
    make_fish("slow", x=250, y=400),
]
```

Nothing else needs to change. The score code already uses `fish.point_value`, and the renderer already uses each fish's color and size.

Test: you should see big slow blue fish (10 pts) and small fast yellow fish (30 pts). Catch one of each and check the score goes up by 10 and 30.

Commit:

```
git add fishing/game/fish.py fishing/game/game_engine.py
git commit -m "Task 2: add slow and fast fish types"
```

Tell me when it's done and we'll do Task 3.

my `__init__` is empty

That's probably `game/__init__.py`, and it's meant to be empty, so leave it alone.

I meant the `__init__` method inside the `GameEngine` class in `game/game_engine.py`. That's where `self.fish_list = [...]` lives:

python

```
class GameEngine:
    def __init__(self):
        self.hook = Hook(...)
        self.fish_list = [          # <- replace this list
            ...
        ]
        self.hooked_fish = None
        self.score = 0
```

If `game_engine.py` itself looks empty, check that you saved the file and that you're in the cloned repo (`03_fishing/fishing/game/`), not another folder.

📄 ↻ 👍 🗨️ ↺ ...

ok done task 3

I haven't given you Task 3 yet. If you wrote it yourself, paste your code and I'll review it. If you meant Task 2 is done, here's Task 3:

1. `game/hook.py` — make `start_cast` ignore calls while a cast is in progress:

python

```
def start_cast(self):
    if self.state != IDLE:
        return
    self.state = CASTING
    self.y = self.surface_y
```

2. `game/game_engine.py`

In `update()`, delete these two lines (they're the auto-loop):

python

```
if self.hook.state == IDLE:
    self.hook.start_cast()
```

Then add this method to `GameEngine`, anywhere above `draw`:

python

```
def cast(self):
    self.hook.start_cast()
```

3. `main.py` — in the event loop, after the `QUIT` check:

python

```
elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE:
    engine.cast()
```

Test: the hook should sit idle until you press SPACE. Mashing SPACE mid-cast should do nothing. It should still return on its own and still score a fish when it surfaces.

Commit:

```
git add fishing/game/hook.py fishing/game/game_engine.py
fishing/main.py
git commit -m "Task 3: player-controlled casting with SPACE"
```

Tell me when it's done and we'll do Task 4, the timer.

📄 🔄 👍 📌 🔄 ...

```
from pkg_resources import resource_stream, resource_exists
pygame 2.6.1 (SDL 2.28.4, Python 3.13.5)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "C:\Users\Lenovo\OneDrive\Desktop\03_fishing\fishing\main.py",
line 39, in <module>
    main()
    ~~~~^^
  File "C:\Users\Lenovo\OneDrive\Desktop\03_fishing\fishing\main.py",
line 27, in main
    engine.cast()
    ^^^^^^^^^^^^^
AttributeError: 'GameEngine' object has no attribute 'cast'
```

`main.py` is calling `engine.cast()`, but Python can't find that method on `GameEngine`. Either it wasn't added, it's indented wrongly, or `game_engine.py` wasn't saved.

Open `game/game_engine.py` and check that `cast` is indented 4 spaces, at the same level as `update` and `draw`:

python

```
class GameEngine:
    def __init__(self):
        ...

    def cast(self):
        self.hook.start_cast()

    def update(self):
        ...
```

A common mistake is pasting it at column 0 or inside `update()`. Save the file and run again.

The `pkg_resources` line at the top is just a harmless pygame warning, so ignore it.

📄 🗨️ 👍 🗨️ 🔄 ...

```
"""
```

GameEngine: owns the hook and the fish, and runs one frame's worth of game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see game/catch.py) that Task 1 asks you to fix.

```
"""
```

```
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
from game.fish import make_fish

class GameEngine:
    def __init__(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            make_fish("slow", x=100, y=180),
            make_fish("fast", x=550, y=230, direction=-1),
            make_fish("slow", x=400, y=280, direction=-1),
            make_fish("fast", x=250, y=340),
            make_fish("slow", x=250, y=400),
        ]
        self.hooked_fish = None
        self.score = 0

    def update(self):
        def cast(self):
            self.hook.start_cast()

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()

    def draw(self, surface, font):
        from game import renderer
        draw_list = list(self.fish_list)
        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)
        renderer.draw_scene(surface, self.hook, draw_list)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

Found it: `def cast` is indented inside `update()`, so it's a function local to `update` and not a method of the class. Move it out so it sits beside `update`:

python

```
        self.hooked_fish = None
        self.score = 0

    def cast(self):
        self.hook.start_cast()

    def update(self):
        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)
        ...
```

So delete the two nested lines at the top of `update()`, and add `cast` above it with 4 spaces of indent. The rest of the file is fine.

     ...

ok fixed but the yellow fishes start dissaperaing after sometime

Most likely they're being caught. In the starter code a caught fish is removed from the pond and never replaced, and yellow ones are worth the most. Check whether the score jumped each time one disappeared. If it did, nothing is broken, but the pond slowly empties, and that will matter for the 30s round.

Fix: respawn a caught fish of the same type off the left edge.

`game/fish.py` — change `make_fish` so each fish remembers its type:

python

```
def make_fish(kind, x, y, direction=1):
    cfg = FISH_TYPES[kind]
    fish = Fish(
        x=x, y=y,
        speed=cfg["speed"] * direction,
        width=cfg["width"], height=cfg["height"],
        point_value=cfg["point_value"], color=cfg["color"],
    )
    fish.kind = kind
    return fish
```

`game/game_engine.py` — add `import random` at the top, then in `update()` replace the scoring block:

python

```
if self.hook.state == IDLE:
    self.score += self.hooked_fish.point_value
    kind = self.hooked_fish.kind
    self.hooked_fish = None
    self.fish_list.append(make_fish(kind, x=-40,
y=random.randint(150, 420)))
```

Commit:

```
git add fishing/game/fish.py fishing/game/game_engine.py
git commit -m "Respawn caught fish so the pond doesn't empty"
```

If the score did **not** change when they vanished, tell me, because then it's a different bug.

📄 💬 👍 🗨️ ↺ ...

```
"""
```

GameEngine: owns the hook and the fish, and runs one frame's worth of game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see game/catch.py) that Task 1 asks you to fix.

```
"""
```

```
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
from game.fish import make_fish
import random

class GameEngine:
    def __init__(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y, max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            make_fish("slow", x=100, y=180),
            make_fish("fast", x=550, y=230, direction=-1),
            make_fish("slow", x=400, y=280, direction=-1),
            make_fish("fast", x=250, y=340),
            make_fish("slow", x=250, y=400),
        ]
        self.hooked_fish = None
        self.score = 0

    def cast(self):
        self.hook.start_cast()

    def update(self):
        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()

    def draw(self, surface, font):
        from game import renderer
        draw_list = list(self.fish_list)
        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)
        renderer.draw_scene(surface, self.hook, draw_list)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

`import random` is in, but the scoring block in `update()` is still the old one.

Replace this:

python

```
if self.hook.state == IDLE:
    self.score += self.hooked_fish.point_value
    self.hooked_fish = None
```

with this:

python

```
if self.hook.state == IDLE:
    self.score += self.hooked_fish.point_value
    kind = self.hooked_fish.kind
    self.hooked_fish = None
    self.fish_list.append(make_fish(kind, x=-40,
y=random.randint(150, 420)))
```

It only works if `make_fish` in `game/fish.py` also sets `fish.kind = kind` before returning. Without that you'll get an `AttributeError: 'Fish' object has no attribute 'kind'` the first time you land a catch. Check that one too.

📄 🔄 👍 🗨️ ↻️ ...

ok its woeking

Good. Make sure the respawn fix is committed first, then on to Task 4.

Replace all of `game/game_engine.py` with this (easier than patching, given the indentation trouble):

python

```
"""
GameEngine: owns the hook and the fish, and runs one frame's worth of
game logic, including the 30-second round timer.
"""

import math
import random

import pygame

from game.hook import Hook, IDLE
from game.fish import make_fish
from game.catch import check_catch
from game import renderer
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y

ROUND_SECONDS = 30

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y,
max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            make_fish("slow", x=100, y=180),
            make_fish("fast", x=550, y=230, direction=-1),
            make_fish("slow", x=400, y=280, direction=-1),
            make_fish("fast", x=250, y=340),
            make_fish("slow", x=250, y=400),
        ]
        self.hooked_fish = None
        self.score = 0
        self.round_start = pygame.time.get_ticks()
        self.game_over = False

    def restart(self):
        if self.game_over:
            self.reset()

    def time_left(self):
        elapsed = (pygame.time.get_ticks() - self.round_start) / 1000
        return max(0.0, ROUND_SECONDS - elapsed)

    def cast(self):
        if not self.game_over:
            self.hook.start_cast()

    def update(self):
        if self.game_over:
            return
```

```

        if self.time_left() <= 0:
            self.game_over = True
            return

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                kind = self.hooked_fish.kind
                self.hooked_fish = None
                self.fish_list.append(make_fish(kind, x=-40,
y=random.randint(150, 420)))
            else:
                caught = check_catch(self.hook, self.fish_list)
                if caught is not None:
                    self.fish_list.remove(caught)
                    self.hooked_fish = caught
                    self.hooked_fish.x = self.hook.x
                    self.hooked_fish.y = self.hook.y
                    self.hook.catch_fish()

    def draw(self, surface, font):
        draw_list = list(self.fish_list)
        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)
        renderer.draw_scene(surface, self.hook, draw_list)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10,
10))

        renderer.draw_text(surface, font, f"Time:
{math.ceil(self.time_left())}", (WIDTH - 110, 10))

        if self.game_over:
            renderer.draw_banner(surface, font, f"Time's up! Final
score: {self.score}")
            msg = "Press R to play again"
            w = font.size(msg)[0]
            renderer.draw_text(surface, font, msg, ((WIDTH - w) // 2,
HEIGHT // 2 + 30))

```

main.py — replace the SPACE handler in the event loop with:

python

```

elif event.type == pygame.KEYDOWN:
    if event.key == pygame.K_SPACE:
        engine.cast()
    elif event.key == pygame.K_r:
        engine.restart()

```

Test: the timer counts down from 30 and the game freezes at 0 with the final score showing. SPACE does nothing then, and R starts a fresh round with the score and timer reset. If the hook is carrying a fish when time runs out, that fish doesn't count, which is my reading of "no further catches".

Commit:

```
git add fishing/game/game_engine.py fishing/main.py
git commit -m "Task 4: 30-second round timer with restart"
```

Tell me when it works, and I'll walk you through the last bit: the "after" video, the `Lab-4` folder, the chat export and the push.

📄 🔄 👍 🗑️ ↺ ...

ok done

Last stretch:

1. **Record the "after" video** (10s). Show: a fish off to the side not getting caught, both fish types, SPACE to cast (mash it mid-cast), a catch scoring, and the timer ending at 0 with the final score. If it doesn't fit in 10s, record the timer end separately.
2. **Check your commits:**

```
git log --oneline
```

You should see a separate commit for each task, plus the respawn fix.

1. **Make the Lab-4 folder** in the repo root and put these in it:
 - `before.mp4` and `after.mp4` (whatever names or format you recorded)
 - a copy of the updated `fishing` folder (or just the changed `.py` files)
 - the chat export (next step)
2. **Export this chat.** Copy it into a doc and save as PDF. For the README's "chat link", also use the chat's share option and put the link in a small text file in `Lab-4`.
3. **Push to your fork:**

```
git remote -v
git add Lab-4
git commit -m "Add Lab-4 deliverables"
git push origin main
```

`git remote -v` should show your username in the URL, not `SETAPESU26`. If it shows `SETAPESU26`, tell me before pushing. And don't open a PR.

Tell me what `git remote -v` shows if you're unsure.

📄 🔄 👍 🗑️ ↺ ...

https://github.com/diran07/PES1UG24AM364_Diran_SE_LAB
so this has to go in my current se folder ?

Let me check how the earlier labs are laid out in that repo.

Yes, that repo is your Lab repo, and the handout says to upload under a `Lab-4` folder in it. It currently has only your three lab PDFs and a README on `main`, no folders yet.

So there are two repos in play:

- **Fishing repo** (your fork of `03_fishing`): the game, with the separate commits for each task. This is where the code work and commit history live.
- `PES1UG24AM364_Diran_SE_LAB`: gets a `Lab-4` folder holding the deliverables.

I'd do it like this:

1. Clone it somewhere outside the fishing folder:

```
git clone https://github.com/diran07/PES1UG24AM364_Diran_SE_LAB.git
cd PES1UG24AM364_Diran_SE_LAB
mkdir Lab-4
```

1. Put into `Lab-4`: the before and after videos, the updated code, and the chat export PDF.
2. Push:

```
git add Lab-4
git commit -m "Add Lab 4 deliverables"
git push origin main
```

One thing to check: the handout says "Have separate commits for each task" and to push the code to your repo. Your task commits are in the fishing clone. Did you fork `03_fishing` to your account, or only clone it from `SETAPESU26`? If you only cloned it, `git push` from there will fail. Tell me which, and we'll sort out where the commit history goes.

📄 🗨️ 👍 🗨️ 🔄 ...

how do i do it step by step tell me

Two parts: first get your commits onto your own GitHub, then put the deliverables in your Lab repo.

Part A: push the game code with its commits

1. In your fishing folder (`...\Desktop\03_fishing`), run:

```
git status
git log --oneline
```

`git status` should say nothing left to commit, and the log should show a separate commit for each task. If anything is uncommitted, commit it first.

1. On GitHub, open `SETAPESU26/03_fishing` and click **Fork**, choosing your account `diran07`.
2. Point your local clone at your fork instead of the original:

```
git remote set-url origin https://github.com/diran07/03_fishing.git
git push origin main
```

Sign in if it asks. Refresh your fork's page on GitHub and check that the commits show up.

Part B: the Lab-4 folder

1. Clone your Lab repo somewhere else, such as the Desktop, not inside the fishing folder:

```
cd %USERPROFILE%\Desktop
git clone https://github.com/diran07/PES1UG24AM364_Diran_SE_LAB.git
```

1. In File Explorer, open the new `PES1UG24AM364_Diran_SE_LAB` folder and create a folder named `Lab-4`.
2. Copy these into `Lab-4`:
 - your before and after videos
 - the `fishing` folder from your game repo (the code)
 - the chat export PDF
 - a small `chat_link.txt` with the share link and your fork URL, `https://github.com/diran07/03_fishing`
3. Push it:

```
cd PES1UG24AM364_Diran_SE_LAB
git add Lab-4
git commit -m "Add Lab 4 deliverables"
git push origin main
```

1. Refresh your Lab repo on GitHub and check that `Lab-4` is there with all the files.

Tell me if any step errors out, and paste the error.



```
Lenovo@LAPTOP-E3HQM294 MINGW64
~/OneDrive/Desktop/PES1UG24AM364_Diran_SE_LAB (main)
$ ls Lab-4/03_fishing
after.mp4 before.mp4 fishing/ README.md
```

Git can't find `Lab-4` in that folder. It's one of these:

- the folder was created somewhere else
- it's spelled differently (e.g. `Lab 4` with a space)
- it's empty (git ignores empty folders)

Run this and paste the output:

```
ls
ls Lab-4
```

If `Lab-4` isn't listed, create it here and copy your files in:

```
mkdir Lab-4
```

Then drag the videos, the `fishing` folder and the chat PDF into `PES1UG24AM364_Diran_SE_LAB\Lab-4` in File Explorer (it's under your Desktop, inside OneDrive). Run `ls Lab-4` again to confirm the files are there, then redo `git add Lab-4`.

📄 ↻ 👍 🗑️ ↺ ...

Now run the push again from inside `PES1UG24AM364_Diran_SE_LAB`:

```
git add Lab-4
git commit -m "Add Lab 4 deliverables"
git push origin main
```

This time `git add` shouldn't error, and the commit should list the files it added. Then refresh your repo on GitHub and check that:

- `Lab-4` is there
- it holds both videos, the `fishing` code folder, the chat PDF and `chat_link.txt`

If a video is over 100 MB, GitHub will reject the push. A 10-second clip should be well under that.

Paste the output if anything looks off.

📄 ↻ 👍 🗑️ ↺ ...

so what do i now need to add in lab 4

Per the handout, `Lab-4` needs three things:

1. **Videos:** the before and after clips (10 seconds each).
2. **Updated code:** the `fishing` folder from your game repo, with all four tasks done.
3. **Chat history:** this conversation exported as a doc or PDF.

I also suggested a small `chat_link.txt` with the share link and your fork URL (https://github.com/diran07/03_fishing). That one's optional, but the game repo's README asks for the chat link.

Which of these do you already have? The chat export is the only one left to make: copy this conversation into a doc, save it as PDF, and drop it in `Lab-4`.



how do i add the update code

Copy the `fishing` folder from your game repo into `Lab-4`:

1. In File Explorer, open your game repo: `Desktop\03_fishing`. You'll see a `fishing` folder inside it (with `main.py` and `game/`).
2. Copy that `fishing` folder.
3. Paste it into `Desktop\PES1UG24AM364_Diran_SE_LAB\Lab-4`.
4. Optional: delete any `__pycache__` folders inside the pasted copy. They're junk.

Check the result looks like this:

```
Lab-4/
  fishing/
    main.py
    requirements.txt
    game/
      catch.py, fish.py, hook.py, game_engine.py, renderer.py,
      __init__.py
```

Then from the Lab repo folder in Git Bash:

```
git add Lab-4
git commit -m "Add Lab 4 updated code"
git push origin main
```

Make sure you copy the `fishing` folder and not the whole `03_fishing` folder, because the whole one would drag its hidden `.git` folder in and cause problems.



ok i added whole fishing file what do i commit

Same commands, from inside the Lab repo folder in Git Bash:

```
git add Lab-4
git commit -m "Add Lab 4 updated code"
git push origin main
```

Run `git status` after the push. It should say `nothing to commit, working tree clean`. Then refresh the repo on GitHub and check that `Lab-4/fishing` is there with `main.py` and the `game` folder.

Videos and the chat PDF go in as a second commit once you have them:

```
git add Lab-4
git commit -m "Add Lab 4 videos and chat history"
git push origin main
```

📄 ↻ 👍 🗨️ ↺ ...

ok next is two videos and chat history link right

Yes. Handout says chat history as a doc/PDF, and the game README asks for a link, so do both to be safe.

1. **Before video:** you already recorded it. Find the file and copy it into `Lab-4`, naming it `before.mp4` (or keep the original extension).
2. **After video:** record 10 seconds now with the finished game. Show a fish off to the side not getting caught, both fish types, casting with SPACE, the timer running out and the final score. Save it as `after.mp4` in `Lab-4`.
3. **Chat history:** copy this whole conversation into a doc and save it as PDF, as `chat_history.pdf` in `Lab-4`.
4. **Chat link:** use this chat's share option, copy the link into `chat_link.txt` in `Lab-4`, and add your fork URL (`https://github.com/diran07/03_fishing`) on a second line.
5. **Push:**

```
git add Lab-4
git commit -m "Add Lab 4 videos and chat history"
git push origin main
```

Then refresh GitHub and check that everything is in `Lab-4`. Make sure you've pushed the game repo to your fork too (Part A from earlier), so your separate task commits are visible there.

📄 ↻ 👍 🗨️ ↺ ...

i cant see the videos pushed

Run these in the Lab repo folder and paste the output:

```
ls Lab-4
git status
git log --oneline -3
```

Likely causes:

- **Videos aren't in Lab-4 yet.** `ls Lab-4` would show it. Copy them in and redo `git add Lab-4`.
- **You didn't commit and push after adding them.** `git status` would show them as untracked or staged. Run `git add Lab-4`, `git commit -m "Add videos"` and `git push origin main`.
- **The push was rejected.** Look for an error like `file exceeds 100 MB`. Screen recordings can be big. If so, trim the clip to 10 seconds, or compress it (e.g. HandBrake), then redo the commit.
- **A `.gitignore` is excluding them.** `git status` won't show the videos at all in that case, even though they're in the folder.

Tell me what the output shows.



```
$ ls Lab-4
git status
git log --oneline -3
03_fishing/
On branch main
Your branch is up to date with 'origin/main'.
```

Found it. You copied the whole `03_fishing` folder, which has its own hidden `.git`, so git treated it as a nested repo (a submodule) and pushed a broken pointer instead of the files. That's why nothing is visible on GitHub. It's fixable.

Run these in the Lab repo folder, one at a time:

1. See what's inside, and whether your videos are in there:

```
ls Lab-4/03_fishing
```

1. Copy just the code out, one level up:

```
cp -r Lab-4/03_fishing/fishing Lab-4/fishing
find Lab-4/fishing -name __pycache__ -exec rm -rf {} +
```

1. If step 1 showed any videos, move them into `Lab-4` itself, in File Explorer or with `mv Lab-4/03_fishing/<name> Lab-4/`.
2. Remove the nested repo from git, then delete the folder:

```
git rm --cached Lab-4/03_fishing
rm -rf Lab-4/03_fishing
```

This only deletes the copy inside `Lab-4`. Your original game folder on the Desktop is untouched.

1. Check, then push:

```
ls Lab-4
git add Lab-4
git commit -m "Fix Lab 4: add code and videos as plain files"
git push origin main
```

`ls Lab-4` should show `fishing` plus your videos (once they're in there), and no `03_fishing`. Paste the output of step 1 if the videos aren't in there.

📄 🗨️ 👍 🗑️ ↺ ...

```
Lenovo@LAPTOP-E3HQM294 MINGW64
~/OneDrive/Desktop/PES1UG24AM364_Diran_SE_LAB (main)
$ git add Lab-4
  git commit -m "Add Lab 4 deliverables"
  git push origin main
fatal: pathspec 'Lab-4' did not match any files
On branch main
Your branch is up to date with 'origin/main'.
```

Both videos are in there, so move them out before deleting anything. Run these in order:

```
cp -r Lab-4/03_fishing/fishing Lab-4/fishing
find Lab-4/fishing -name __pycache__ -exec rm -rf {} +
mv Lab-4/03_fishing/after.mp4 Lab-4/
mv Lab-4/03_fishing/before.mp4 Lab-4/
git rm --cached Lab-4/03_fishing
rm -rf Lab-4/03_fishing
ls Lab-4
```

`ls Lab-4` should show exactly `after.mp4`, `before.mp4` and `fishing`, with no `03_fishing`. Then:

```
git add Lab-4
git commit -m "Fix Lab 4: add code and videos as plain files"
git push origin main
```

Add the chat PDF and `chat_link.txt` as a separate commit later.

📄 🔄 👍 🗨️ ↻ ...

ok lets add this now

Assuming the fix pushed and you mean the chat history, here's how:

1. **PDF:** select the whole conversation on the page, copy it, paste into a Word or Google Doc, and save as PDF named `chat_history.pdf`. Plain monospace text is fine; it doesn't need styling.
2. **Link:** use this chat's share option, copy the link, and make `chat_link.txt` with the link on line one and `https://github.com/diran07/03_fishing` on line two.
3. Put both in `Lab-4`, then:

```
git add Lab-4
git commit -m "Add Lab 4 chat history"
git push origin main
```

If you meant the commands from my last message instead, paste the output of `ls Lab-4` and I'll check it.

📄 🔄 👍 🗨️ ↻ ...

tell me again what to do

Here's the whole remaining list, in order. Run the git commands in Git Bash inside `PES1UG24AM364_Diran_SE_LAB`.

1. Fix the Lab-4 folder (skip if you already ran this)

```
cp -r Lab-4/03_fishing/fishing Lab-4/fishing
find Lab-4/fishing -name __pycache__ -exec rm -rf {} +
mv Lab-4/03_fishing/after.mp4 Lab-4/
mv Lab-4/03_fishing/before.mp4 Lab-4/
git rm --cached Lab-4/03_fishing
rm -rf Lab-4/03_fishing
ls Lab-4
```

`ls Lab-4` should show `after.mp4`, `before.mp4` and `fishing`.

2. Push it

```
git add Lab-4
git commit -m "Fix Lab 4: add code and videos as plain files"
git push origin main
```

3. Make the chat files and put them in `Lab-4`

- `chat_history.pdf`: copy this conversation into a doc and save it as PDF.
- `chat_link.txt`: this chat's share link on line one, https://github.com/diran07/03_fishing on line two.

4. Push them

```
git add Lab-4
git commit -m "Add Lab 4 chat history"
git push origin main
```

5. Check on GitHub. `Lab-4` should contain `after.mp4`, `before.mp4`, `fishing/`, `chat_history.pdf` and `chat_link.txt`.

Tell me which step you're on if anything errors.



no i meant the chat history thing step by step

Step by step:

1. **Copy the chat.** Scroll to the top of this conversation, click at the start of the first message, then shift-click at the end of the last one so everything is selected. Copy with `Ctrl+C`.
2. **Paste into a doc.** Open Word or Google Docs and paste as plain text with `Ctrl+Shift+V`. Set the font to Consolas or Courier New, and leave out any extra styling.
3. **Save as PDF.** In Word use File > Save As > PDF. In Google Docs use File > Download > PDF. Name it `chat_history.pdf`.
4. **Move it** into `PES1UG24AM364_Diran_SE_LAB\Lab-4`.
5. **Get the share link.** Use the share option for this chat and copy the link.
6. **Make** `chat_link.txt`. Open Notepad and put the share link on line one and `https://github.com/diran07/03_fishing` on line two. Save it as `chat_link.txt` inside `Lab-4`. In the Save dialog, set "Save as type" to All Files so it doesn't become `chat_link.txt.txt`.
7. **Push:**

```
git add Lab-4
git commit -m "Add Lab 4 chat history"
git push origin main
```

1. **Check GitHub.** `chat_history.pdf` and `chat_link.txt` should be in `Lab-4`.

Do the export last, right before you copy, so the PDF includes everything up to your final messages.



```
```python
"""
```

GameEngine: owns the hook and the fish, and runs one frame's worth of game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see `game/catch.py`) that Task 1 asks you to fix.

```
"""
```

```
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y,
MAX_DEPTH_Y
from game.fish import make_fish

class GameEngine:
 def __init__(self):
 self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y,
max_depth_y=MAX_DEPTH_Y, speed=5)
 self.fish_list = [
 make_fish("slow", x=100, y=180),
 make_fish("fast", x=550, y=230, direction=-1),
 make_fish("slow", x=400, y=280, direction=-1),
 make_fish("fast", x=250, y=340),
 make_fish("slow", x=250, y=400),
]
 self.hooked_fish = None
 self.score = 0

 def update(self):
 def cast(self):
 self.hook.start_cast()

 self.hook.update()

 for fish in self.fish_list:
 fish.update(WIDTH)

 if self.hooked_fish is not None:
 self.hooked_fish.x = self.hook.x
 self.hooked_fish.y = self.hook.y
 if self.hook.state == IDLE:
 self.score += self.hooked_fish.point_value
 self.hooked_fish = None
 else:
 caught = check_catch(self.hook, self.fish_list)
 if caught is not None:
 self.fish_list.remove(caught)
 self.hooked_fish = caught
 self.hooked_fish.x = self.hook.x
 self.hooked_fish.y = self.hook.y
 self.hook.catch_fish()

 def draw(self, surface, font):
 from game import renderer
 draw_list = list(self.fish_list)
 if self.hooked_fish is not None:
 draw_list.append(self.hooked_fish)
 renderer.draw_scene(surface, self.hook, draw_list)
 renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

Most likely they're being caught. In the starter code a caught fish is removed from the pond and never replaced, and yellow ones are worth the most. Check whether the score jumped each time one disappeared. If it did, nothing is broken, but the pond slowly empties, and that will matter for the 30s round.

Fix: respawn a caught fish of the same type off the left edge.

`**`game/fish.py`**` — change ``make_fish`` so each fish remembers its type:

```
```python
def make_fish(kind, x, y, direction=1):
    cfg = FISH_TYPES[kind]
    fish = Fish(
        x=x, y=y,
        speed=cfg["speed"] * direction,
        width=cfg["width"], height=cfg["height"],
        point_value=cfg["point_value"], color=cfg["color"],
    )
    fish.kind = kind
    return fish
```
```

`**`game/game_engine.py`**` — add ``import random`` at the top, then in ``update()`` replace the scoring block:

```
```python
if self.hook.state == IDLE:
    self.score += self.hooked_fish.point_value
    kind = self.hooked_fish.kind
    self.hooked_fish = None
    self.fish_list.append(make_fish(kind, x=-40, y=random.randint(150, 420)))
```
```

`**Commit:**`

```
```
git add fishing/game/fish.py fishing/game/game_engine.py
git commit -m "Respawn caught fish so the pond doesn't empty"
```
```

If the score did `**not**` change when they vanished, tell me, because then it's a different bug.

```
```python
"""
```

GameEngine: owns the hook and the fish, and runs one frame's worth of game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see `game/catch.py`) that Task 1 asks you to fix.

```
"""
```

```
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y,
MAX_DEPTH_Y
from game.fish import make_fish
import random
```

```
class GameEngine:
    def __init__(self):
        self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y,
max_depth_y=MAX_DEPTH_Y, speed=5)
        self.fish_list = [
            make_fish("slow", x=100, y=180),
            make_fish("fast", x=550, y=230, direction=-1),
            make_fish("slow", x=400, y=280, direction=-1),
            make_fish("fast", x=250, y=340),
            make_fish("slow", x=250, y=400),
        ]
        self.hooked_fish = None
        self.score = 0

    def cast(self):
        self.hook.start_cast()

    def update(self):

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()

    def draw(self, surface, font):
        from game import renderer
        draw_list = list(self.fish_list)
        if self.hooked_fish is not None:
```

```
draw_list.append(self.hooked_fish)
renderer.draw_scene(surface, self.hook, draw_list)
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

...

Good. Make sure the respawn fix is committed first, then on to Task 4.

Replace all of `game/game\_engine.py` with this (easier than patching, given the indentation trouble):

```
```python
"""
GameEngine: owns the hook and the fish, and runs one frame's worth of
game logic, including the 30-second round timer.
"""

import math
import random

import pygame

from game.hook import Hook, IDLE
from game.fish import make_fish
from game.catch import check_catch
from game import renderer
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y

ROUND_SECONDS = 30

class GameEngine:
 def __init__(self):
 self.reset()

 def reset(self):
 self.hook = Hook(x=WIDTH / 2, surface_y=SURFACE_Y,
 max_depth_y=MAX_DEPTH_Y, speed=5)
 self.fish_list = [
 make_fish("slow", x=100, y=180),
 make_fish("fast", x=550, y=230, direction=-1),
 make_fish("slow", x=400, y=280, direction=-1),
 make_fish("fast", x=250, y=340),
 make_fish("slow", x=250, y=400),
]
 self.hooked_fish = None
 self.score = 0
 self.round_start = pygame.time.get_ticks()
 self.game_over = False

 def restart(self):
 if self.game_over:
 self.reset()

 def time_left(self):
 elapsed = (pygame.time.get_ticks() - self.round_start) / 1000
 return max(0.0, ROUND_SECONDS - elapsed)
```

```

def cast(self):
 if not self.game_over:
 self.hook.start_cast()

def update(self):
 if self.game_over:
 return
 if self.time_left() <= 0:
 self.game_over = True
 return

 self.hook.update()

 for fish in self.fish_list:
 fish.update(WIDTH)

 if self.hooked_fish is not None:
 self.hooked_fish.x = self.hook.x
 self.hooked_fish.y = self.hook.y
 if self.hook.state == IDLE:
 self.score += self.hooked_fish.point_value
 kind = self.hooked_fish.kind
 self.hooked_fish = None
 self.fish_list.append(make_fish(kind, x=-40, y=random.randint(150, 420)))
 else:
 caught = check_catch(self.hook, self.fish_list)
 if caught is not None:
 self.fish_list.remove(caught)
 self.hooked_fish = caught
 self.hooked_fish.x = self.hook.x
 self.hooked_fish.y = self.hook.y
 self.hook.catch_fish()

def draw(self, surface, font):
 draw_list = list(self.fish_list)
 if self.hooked_fish is not None:
 draw_list.append(self.hooked_fish)
 renderer.draw_scene(surface, self.hook, draw_list)
 renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
 renderer.draw_text(surface, font, f"Time: {math.ceil(self.time_left())}",
 (WIDTH - 110, 10))

 if self.game_over:
 renderer.draw_banner(surface, font, f"Time's up! Final score: {self.score}")
 msg = "Press R to play again"
 w = font.size(msg)[0]
 renderer.draw_text(surface, font, msg, ((WIDTH - w) // 2, HEIGHT // 2 + 30))
 ...

`main.py` — replace the SPACE handler in the event loop with:

```

```

```python
elif event.type == pygame.KEYDOWN:
if event.key == pygame.K_SPACE:
engine.cast()
elif event.key == pygame.K_r:
engine.restart()
```

```

**\*\*Test:\*\*** the timer counts down from 30 and the game freezes at 0 with the final score showing. SPACE does nothing then, and R starts a fresh round with the score and timer reset. If the hook is carrying a fish when time runs out, that fish doesn't count, which is my reading of "no further catches".

**\*\*Commit:\*\***  
 ```

```

git add fishing/game/game_engine.py fishing/main.py
git commit -m "Task 4: 30-second round timer with restart"
```

```

Tell me when it works, and I'll walk you through the last bit: the "after" video, the `Lab-4` folder, the chat export and the push.








```

$ ls Lab-4
git status
git log --oneline -3
03_fishing/
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
 (use "git add <file>..." to update what will be committed)
 (use "git restore <file>..." to discard changes in working directory)
 (commit or discard the untracked or modified content in submodules)
 modified: Lab-4/03_fishing (untracked content)

no changes added to commit (use "git add" and/or "git commit -a")
9df5ade (HEAD -> main, origin/main, origin/HEAD) Add Lab 4 updated code
9692293 Delete PES1UG24AM364_SELAB4
18bb7c7 Create PES1UG24AM364_SELAB4

Lenovo@LAPTOP-E3HQM294 MINGW64
~/OneDrive/Desktop/PES1UG24AM364_Diran_SE_LAB (main)

```

Found it. You copied the whole `o3\_fishing` folder, which has its own hidden `.git`, so git treated it as a nested repo (a submodule) and pushed a broken pointer instead of the files. That's why nothing is visible on GitHub. It's fixable.

Run these in the Lab repo folder, one at a time:

1. See what's inside, and whether your videos are in there:

```
...
```

```
ls Lab-4/o3_fishing
```

```
...
```

2. Copy just the code out, one level up:

```
...
```

```
cp -r Lab-4/o3_fishing/fishing Lab-4/fishing
```

```
find Lab-4/fishing -name __pycache__ -exec rm -rf {} +
```

```
...
```

3. If step 1 showed any videos, move them into `Lab-4` itself, in File Explorer or with `mv Lab-4/o3\_fishing/<name> Lab-4/`.

4. Remove the nested repo from git, then delete the folder:

```
...
```

```
git rm --cached Lab-4/o3_fishing
```

```
rm -rf Lab-4/o3_fishing
```

```
...
```

This only deletes the copy inside `Lab-4`. Your original game folder on the Desktop is untouched.

5. Check, then push:

```
...
```

```
ls Lab-4
```

```
git add Lab-4
```

```
git commit -m "Fix Lab 4: add code and videos as plain files"
```

```
git push origin main
```

```
...
```

`ls Lab-4` should show `fishing` plus your videos (once they're in there), and no `o3\_fishing`. Paste the output of step 1 if the videos aren't in there.

     ...